

1.Node와 조건부 분기 실행 및 Trace 분석

Node 개념 — Agent 실행 단위

Node란 무엇인가?

Agent 실행 흐름에서 하나의 책임을 맡는 처리 단계입니다. State를 입력으로 받아 처리하고, 결과를 다시 State에 추가한 뒤 다음 Node로 전달합니다.

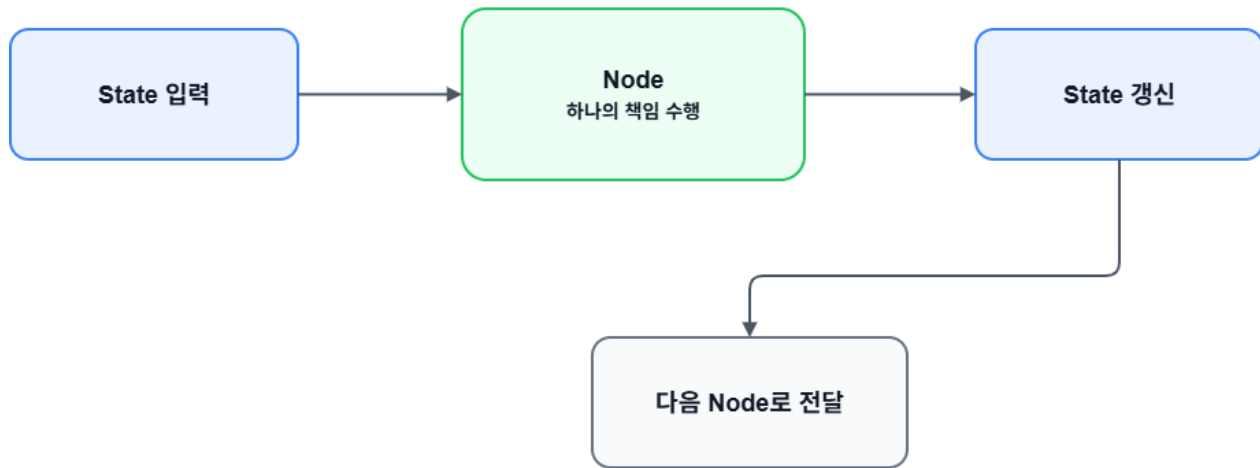
RAG Agent의 Node 구성

- 사용자 질문 분석 — 설비 ID, 알람 코드 추출
- 문서 검색 — RAG 근거 후보 확보
- 답변 초안 생성 — 검색 근거 기반 응답 작성
- 근거 검증 — grounding 상태 확인
- 질문 재작성 — 근거 부족 시 재검색 질의 변환

 Node는 단순 함수가 아닙니다. State를 입력받아 처리하고 다음 단계로 넘기는 **Agent 실행 단위**입니다. 각 Node는 독립적으로 하나의 일을 수행하지만, State를 통해 이전 단계의 처리 문맥을 공유합니다.

Node의 설계 의미

Node는 단순 함수가 아니라,
State를 입력받아 하나의 책임을 수행하고
다음 단계로 넘기는 Agent 실행 단위입니다.



핵심: Node는 Agent 흐름에서 책임을 분리하고,
State를 통해 이전 단계의 처리 문맥을 이어갑니다.

RAG Graph 실행 파일 — 핵심 구조

src/day2/langgraph_rag_graph_runner.py는 RAG Agent v1의 Node 실행 순서와 조건부 분기를 정의하고, 실행 결과를 Trace로 남기는 핵심 파일입니다.

이 파일의 역할

State를 입력으로 받아 여러 Node를 순서대로 실행합니다. 문서 검색 → 답변 초안 생성 → 근거 충분성 확인의 흐름을 제어합니다.

조건부 분기 포함

근거가 부족할 경우 질문을 재작성하고 검색을 반복하는 분기 흐름이 포함되어 있습니다. 이 분기가 Agent의 품질 관리 구조입니다.

설계 관점

검색 결과가 어떻게 답변 생성으로 연결되고, 근거가 부족할 때 어떤 분기로 이동하는지 확인하는 기준 파일입니다.

 langgraph_rag_graph_runner.py는 RAG Agent v1의 State 기반 실행 흐름과 조건부 분기를 확인하는 기준 파일입니다.

주요 Node 흐름 — 역할과 설계 관점

Node 이름	역할	설계 관점
parse_query_node	사용자 질문 분석	설비 ID, 알람 코드 등 검색·Tool 호출에 필요한 조건 추출
retrieve_docs_node	문서 검색	RAG 근거 후보를 확보하는 단계
generate_answer_node	답변 초안 생성	검색 근거를 바탕으로 응답 초안 생성
verify_grounding_node	근거 확인	답변이 검색 근거와 연결되어 있는지 검토
query_rewrite_node	질문 재작성	근거 부족 시 재검색 가능한 질의로 변환

각 Node는 하나의 책임을 맡고 State를 갱신합니다. 역할을 분리하면 검색, 답변 생성, 근거 검증, 재검색 흐름을 독립적으로 점검할 수 있습니다. 각 Node는 다음 Node가 사용할 정보를 State에 명확히 남깁니다.

반복 검토·수정 구조



분기 제어 변수

- `grounding_status` — 답변이 검색 근거를 충분히 반영했는지 판단
- `needs_rewrite` — 재검색 흐름으로 이동할지 결정하는 분기 조건

설계 관점

조건부 분기는 Agent가 근거 없는 답변을 생성하지 않도록 제어하는 품질 관리 구조입니다. 근거 부족 상황에서 무리하게 답변하지 않고 재검색 또는 fallback 흐름으로 이동합니다.

RAG Graph 실행 — PowerShell 명령어

명령어는 C:\work\manufacturing_agent_project\ 루트 위치에서 실행합니다.

실행 위치 확인

C:\work\manufacturing_agent_project\ 가 루트 폴더입니다.
PowerShell에서 해당 경로로 이동한 뒤 실행합니다.

실행 명령어

```
uv run python src/day2/langgraph_rag_graph_runner.py
```

실행 목적

State가 Node 사이를 이동하면서
검색 결과, 답변 초안, 근거 검증 상태가 어떻게 추가되는지 확인합니다.



이 실행은 RAG Graph 흐름을 실행하고, Node 실행 결과와 Trace를 확인할 수 있게 합니다.

산출물 확인 — RAG Graph Trace

산출물 위치

outputs/day2/langgraph_rag_trace.md

RAG Graph 실행 흐름을 정리한 Markdown 산출물입니다.

Trace에서 확인할 내용

- 어떤 Node가 어떤 순서로 실행되었는가
- 검색 결과가 충분했는가
- 질문 재작성 여부가 있었는가
- Node 실행 순서와 State 변화 흐름
- 근거 검증 결과 (grounding 상태)

- ✔ langgraph_rag_trace.md는 단순 실행 로그가 아닙니다. Node 실행 순서, State 변화, 검색 근거, 근거 검증 결과를 확인하는 Trace 기반 실행 검토 자료입니다. 이 Trace를 확인해야 최종 결과가 어떤 근거와 실행 흐름을 거쳐 생성되었는지 설명할 수 있습니다.

Trace의 위치

Trace는 2일차에서 실행 흐름을 확인하는 검토 자료이며,
본격적인 실행 품질 평가는 4일차에서 다룹니다.



Day2에서는 Trace를 “실행 검토 자료”로 사용하고,
Day4에서는 Trace를 기반으로 응답 품질과 안정성을 평가합니다.

Trace 결과 해석 — 실행 검토 관점

1 Node 실행 순서 확인

어떤 Node가 실행되었는지, 각 단계에서 State가 어떻게 바뀌었는지 확인합니다.

2 근거 충분성 검토

검색 근거가 충분했는지, 질문 재작성이나 재검색이 있었는지 확인합니다.

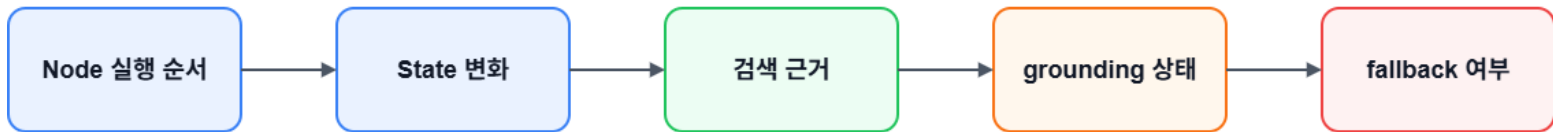
3 실행 흐름 연결 확인

최종 답변이 질문 분석 → 문서 검색 → 답변 생성 → 근거 확인 흐름을 거쳐 만들어졌음을 검증합니다.

❏ 이 Trace 구조는 3일차 MCP Tool 호출 Trace와 4일차 실행 품질 평가의 기반이 됩니다. 다음 장에서는 Day2 RAG Agent v1 최종 실행 파일을 통해 검색 근거, State, Trace가 반영된 최종 결과를 생성합니다.

Trace 해석 기준

Trace에서는 실행 순서뿐 아니라 근거와 상태 변화까지 함께 확인합니다.



확인 질문

- ① 어떤 Node가 실행되었는가?
- ② 검색 근거가 충분했는가?
- ③ 최종 답변이 근거와 연결되는가?
- ④ fallback 또는 재검색이 있었는가?

2.Day2 RAG Agent v1 최종 실행과 근거 기반 결과 해석

최종 실행 파일 — day2_rag_agent_v1.py

src/day2/day2_rag_agent_v1.py는 2일차 RAG 실습의 최종 실행 파일입니다. 사용자 질문을 준비하고, RAG Graph 실행 함수를 호출하며, 실행 결과 State를 받아 검색 근거와 답변 결과를 Markdown 결과로 저장합니다.

목적

단순히 결과 문서를 만드는 것이 아닙니다. 검색 근거와 State/Trace 흐름이 최종 답변에 어떻게 반영되었는지 확인하는 것입니다.

통합 실행 구조

검색 모듈과 Graph 실행 모듈을 호출하고, 그 결과를 최종 실행 결과로 정리하는 통합 실행 파일입니다.

설계 관점

문서 검색, State 흐름, Node 실행, 근거 검증 결과를 통합해 2일차 최종 실행 결과를 확인하게 해주는 파일입니다.

역할 분리 구조 — 파일별 책임

기능	담당 파일	역할
문서 검색	rag_search.py	질문과 관련 있는 근거 후보 검색
Node 실행	langgraph_rag_graph_runner.py	검색·답변 생성·근거 검증 흐름 실행
State 구조 관리	graph_state.py	사용자 질문, 검색 결과, 답변 초안, Trace 저장 구조 정의
최종 결과 정리	day2_rag_agent_v1.py	실행 결과 State를 받아 Markdown 결과로 정리

 역할을 분리하면 문서 검색, Node 실행, State 관리, 결과 정리를 각각 따로 점검할 수 있습니다. 최종 파일은 여러 실행 결과를 모아 단순 보고서로 정리하는 파일이 아니라, 2일차 RAG Agent v1의 전체 실행 흐름을 통합하는 파일입니다.

최종 실행 — PowerShell 명령어

명령어는 C:\work\manufacturing_agent_project\ 폴더의 루트 위치, 즉 src 폴더가 보이는 위치에서 실행합니다.

실행 위치

PowerShell에서 프로젝트 폴더로 이동한 뒤 실행합니다.

실행 명령어

```
uv run python src/day2/day  
2_rag_agent_v1.py
```

실행 결과

Day2 RAG Agent v1 최종 흐름을 실행하고, 결과 Markdown 파일을 저장합니다.



이 실행은 2일차 RAG Agent v1의 최종 State와 근거 기반 답변 결과를 생성하는 단계입니다.

산출물 확인 — 최종 실행 결과

산출물 위치

outputs/day2/day2_rag_agent_v1_result.md

Day2 RAG Agent v1의 최종 실행 결과 파일입니다.

이 파일에서 확인할 내용

- 사용자 질문 및 설비 ID, 알람 코드
- 검색 근거 (Top-3 근거 후보)
- 답변 초안 및 최종 답변
- grounding 상태 (근거 연결 여부)
- Trace 요약 (Node 실행 흐름)

✅ 이 파일은 답변 문장뿐 아니라 검색 근거, grounding 상태, Trace 흐름을 함께 확인하는 기준 결과입니다. 3일차에는 이 RAG 검색 기능을 search_manual MCP Tool로 분리합니다.

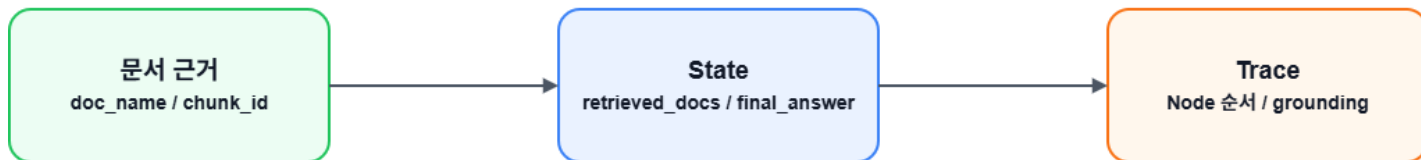
최종 결과 해석 — 항목별 확인 포인트

결과 항목	의미	확인 포인트
사용자 질문	입력된 장애 문의	질문이 명확한가?
설비 ID / 알람 코드	대상 설비·알람	EQP-EV-03, ALM-TEMP-402 등이 추출되었는가?
검색 근거	Agent가 참고한 문서 조각	답변이 어떤 문서를 근거로 했는가?
검색 문서 출처	근거 문서 위치	doc_name, chunk_id, section_title이 확인되는가?
원인 후보 / 1차 조치	가능한 원인 및 현장 점검 항목	단정 표현이 아닌 가능성으로 표현되었는가?
grounding 상태	근거 사용 상태	답변이 문서 근거와 연결되어 있는가?
Trace 요약	실행 흐름 기록	어떤 Node를 거쳐 결과가 생성되었는가?
fallback 여부	안전 응답 여부	근거 부족 시 무리하게 답변하지 않았는가?

 최종 결과는 문장이 자연스러운지보다, 어떤 문서 근거를 사용했고 그 근거가 State와 Trace에 남아 있는지 확인하는 것이 중요합니다.

최종 결과 판단 기준

최종 결과는 답변 문장의 자연스러움보다,
어떤 문서 근거를 사용했고 그 근거가 State와 Trace에 남아 있는지를 중심으로 검토합니다.



판단 기준: “답변이 좋아 보이는가?”보다
“근거가 무엇이며 실행 흐름에 남아 있는가?”를 확인합니다.

3.Streamlit 보조 화면과 선택/확장 실습

Streamlit 위치

Streamlit의 위치

Streamlit은 RAG Agent 실행 흐름을 화면에서 확인하기 위한 보조 UI입니다. 기본 실습의 기준은 CLI 실행 결과와 outputs/day2 산출물입니다.

실행 명령어

```
uv run streamlit run src/day2/day2_rag_agent_streamlit_app.py
```

Streamlit에서 확인 가능한 내용

- 사용자 질문 입력 및 검색 결과 확인
- State 변화 시각화
- 최종 결과 확인 (웹 화면)

운영 관점

실행이 어렵다면 UI 디버깅에 오래 머무르지 않고 CLI 결과와 outputs/day2 산출물로 진행합니다. Streamlit은 기본 수업을 대체하는 도구가 아닙니다.

 Streamlit 실행 전에는 requirements.txt 설치가 완료되어 있어야 합니다.

Chroma Vector DB

실습 파일

`src/day2/chroma_index_builder.py`

chunk 결과를 Chroma Vector DB에 저장하는 선택 실습 파일입니다.

실행 명령어

```
uv run python src/day2/chroma_index_builder.py
```

산출물

`outputs/day2/chroma_build_result.md`

Chroma의 역할

- 검색 가능한 문서 벡터를 저장
- 사용자 질의와 의미적으로 가까운 문서 chunk 검색
- embedding — 문장의 의미를 벡터로 표현하여 의미 기반 검색 가능

운영 관점

2일차 기본 목표는 Vector DB 구축 자체가 아닙니다. 기본 수업의 중심은 Top-3 검색 결과가 State와 Trace에 어떻게 연결되는지 이해하는 것입니다. 환경 문제가 있으면 강사 데모 또는 Saved Result Review로 대체할 수 있습니다.

실습 방

1

embedding 기반 의미 검색

목적: 의미 기반 검색 구조 이해

확인 내용: 키워드가 달라도 유사 의미의 chunk를 찾는지 확인

2

키워드 검색 vs Vector DB 비교

목적: 검색 방식별 장단점 비교

확인 내용: 정확 조건 검색과 의미 검색의 결과 차이 비교

3

Streamlit 화면 확인

목적: 실행 흐름 시각화

확인 내용: State, 검색 결과, 최종 답변이 어떻게 연결되는지 확인

i 확장 실습의 목적은 새로운 도구를 많이 실행하는 것이 아닙니다. 검색 방식에 따라 근거 후보가 어떻게 달라지고, 그 결과가 State와 최종 답변에 어떤 영향을 주는지 비교하는 것입니다. 기본 수업을 모두 마친 뒤 여유가 있을 때 다룹니다.

4.2일차 산출물 정리와 3일차 MCP 연결

2일차 산출물 목록 — 기준 자료

구분	산출물	의미	다음 단계 역할
필수	document_load_result.md	문서 로드 결과	RAG 대상 문서 입력 구조 확인
필수	chunk_preview.json	chunk와 metadata 미리보기	검색 가능한 지식 단위 구조 확인
필수	chunk_build_result.md	chunk 생성 결과	문서가 검색 가능한 지식 단위로 분리되었는지 확인
필수	rag_test_results.md	Top-3 검색 결과	search_manual Tool의 검색 결과 구조 기반
필수	state_demo_result.md	State 변화 결과	Node 간 전달되는 업무 처리 문맥 확인
필수	langgraph_rag_trace.md	RAG Graph 실행 Trace	Node 실행 순서, State 변화, 근거 검증 흐름
필수	day2_rag_agent_v1_result.md	최종 RAG Agent 결과	search_manual Tool 분리 전 기준 결과
선택	chroma_build_result.md	Chroma 인덱스 생성 결과	Vector DB 선택 실습 결과

👉 2일차 산출물은 단순 결과물이 아닙니다. 문서 입력, chunk, metadata, 검색 결과, State, Trace, 최종 답변이 어떻게 연결되는지 확인하는 기준 자료이며, 3일차 search_manual MCP Tool 설계와 4일차 Trace 품질 평가의 기반이 됩니다.

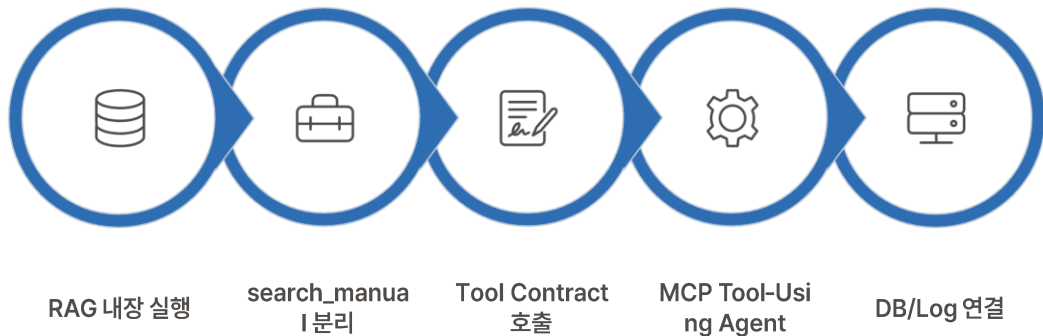
산출물의 설계 의미

Streamlit과 Chroma는 기본 실습을 대체하지 않고,
CLI 실행 결과와 outputs/day2 산출물을 확인한 뒤 사용하는 보조·선택 요소입니다.



환경 문제가 발생하면 UI/설치 디버깅에 오래 머무르지 않고,
Saved Result Review 방식으로 진행합니다.

3일차 연결 — MCP Tool-Using Agent



2일차에서 완성한 것

- RAG 검색 기능을 Agent 내부 실행 흐름으로 구성
- 검색 결과를 State에 저장하고 근거 기반 답변 생성
- langgraph_rag_trace.md로 Node 실행 흐름 확인

3일차에서 확장할 것

- RAG 검색 기능을 search_manual Tool로 분리
- Tool Contract 기반으로 호출 가능한 구조로 전환
- 제조 DB/Log Tool 연결 — 설비 상태, 알람 이력, 품질 지표 조회
- MCP 방식으로 호출하는 구조로 확장

Day2 RAG Agent v1은 3일차 MCP Tool-Using Agent의 문서 검색 기반입니다. 오늘 만든 RAG 검색 기능을 search_manual Tool로 분리하고, DB/Log Tool과 함께 MCP 방식으로 호출하는 구조로 확장합니다.